

FINAL PROJECT

By Jean Claude NIYOMUGABO

AGIRITECHAT: AGENT-BASED AI DECISION SUPPORT FOR SUSTAINABLE AGRICULTURE

University of Nebraska–Lincoln

Department of Biological Systems Engineering

AGST 492: Agentic AI for Workflow Automation

Instructor: Dr. Santosh Kumar Pitla

Date: May 1, 2026

1. Introduction

Agriculture remains a critical sector globally, supporting the livelihoods of more than 2.5 billion people. Despite its importance, access to timely and reliable agricultural advisory services remains a major challenge. In maize and soybean production systems, farmers frequently face issues such as pests, diseases, soil fertility constraints, and nutrient deficiencies, yet often lack immediate access to expert support.

Traditional agricultural extension systems are limited in scale and responsiveness. In many developing regions, the ratio of extension agents to farmers can exceed 1:3,000 (Swanson & Davis, 2014). Even in developed systems like Nebraska, farmers depend on seasonal consultations and fragmented digital resources. Key management decisions are often made with incomplete information.

This project introduces AgiriteChat, an agent-based AI assistant designed to support maize and soybean farmers in Nebraska and Rwanda's Eastern Province. The system integrates a curated 101-entry knowledge base, a LangGraph state machine with 7 nodes and confidence gating, semantic retrieval using sentence-transformers, Groq-powered generative AI (Llama 3.3 70B for text, Llama 4 Scout 17B for vision, Whisper for voice), multilingual support in four languages, weather display, farmer profile personalization, and source-separated answers that clearly distinguish verified knowledge base content from AI-generated guidance. The system is deployed at agiritechat.streamlit.app, demonstrating that scalable AI tools can be built at zero cost using open-source models and free-tier infrastructure.

2. Objectives

The overall objective is to design and develop an agent-based AI system that supports maize and soybean farmers by providing structured and context-aware guidance for crop problem diagnosis and management. The specific objectives are:

1. Design and implement a LangGraph-based agent workflow with 7 nodes: classify, greet, retrieve, check_confidence, clarify, refuse, and synthesize.
2. Develop a domain-specific knowledge base with 101 curated entries covering pests, diseases, nutrient deficiencies, soil management, planting, harvest, storage, and general agronomy for Nebraska and Rwanda Eastern Province.
3. Implement semantic retrieval using sentence-transformers (all-MiniLM-L6-v2) with cosine similarity for meaning-based matching.
4. Integrate confidence gating with explicit thresholds (0.55 high, 0.35 low) that routes queries to synthesis, clarification, or honest refusal.
5. Build a vision pipeline using Groq Vision (Llama 4 Scout 17B) that extracts symptoms from photos without diagnosing, then feeds observations to the retrieval agent.
6. Add voice input via Groq Whisper (whisper-large-v3-turbo) for low-literacy users who may find text input challenging.
7. Integrate weather display using Open-Meteo free API showing current conditions and 3-day forecast for the farmer's region.
8. Support four languages (English, Kiswahili, Français, Kinyarwanda) with native generation, not post-translation.
9. Implement source separation that clearly labels verified knowledge base content vs. AI-generated guidance in every response.

10. Deploy as a functional, publicly accessible prototype on Streamlit Cloud with zero-cost infrastructure.

3. System Architecture

AgiriteChat is implemented as a modular Python application with 8 core files:

- app.py (2,100 lines): Streamlit UI with editorial-style landing page, tabs for Ask/Photo/Library, Quick starts presets, voice input (Whisper), answer rendering with source separation badges, feedback widgets, and full four-language translation table.
- agent.py (620 lines): LangGraph state machine with 7 nodes. Classify detects greetings, crop-name-only inputs, and agricultural questions. Greet handles non-agricultural inputs conversationally. Retrieve performs semantic search. Check_confidence routes based on retrieval scores and input clarity. Clarify asks for symptoms. Refuse honestly says “I don’t know.” Synthesize generates grounded 5-section advisories with source attribution.
- llm.py (254 lines): Groq SDK wrapper providing text generation (Llama 3.3 70B), JSON structured output, vision analysis (Llama 4 Scout 17B), and voice transcription (Whisper large-v3-turbo).
- retrieval.py (121 lines): Sentence-transformer embeddings (all-MiniLM-L6-v2) with NumPy cosine similarity. Replaced ChromaDB due to deployment dependency conflicts on Streamlit Cloud.
- vision.py: OpenCV quality checks (blur, brightness) followed by Groq Vision symptom extraction. The vision model describes symptoms; the KB provides diagnosis. This separation prevents hallucination on disease identification.

- `weather.py` (139 lines): Open-Meteo API integration. Matches farmer's region to coordinates for 14 East African and US locations. No API key required.
- `feedback.py`: SQLite interaction logging with session tracking, confidence recording, and thumbs up/down feedback for continuous improvement.
- `knowledge_base.json`: 101 curated entries with structured fields for question, answer, crop, category, symptoms, growth_stage, region, source, and confidence. Covers 98 unique sources from UNL Extension, CIMMYT, IITA, RAB, FAO, and ICIPE.

4. Agentic AI Design

AgiriteChat demonstrates genuine agentic AI behavior through its multi-step decision pipeline, consistent with the definition of AI agents as systems that perceive their environment, reason about it, and take autonomous action (Russell & Norvig, 2021). Each node in the pipeline makes an independent decision that affects downstream behavior:

The classify node determines whether the input is agricultural, a greeting, or a crop name without symptoms. This uses both regex pattern matching (for greetings) and LLM-based classification (for agricultural categorization). The system detects greetings in all four supported languages.

The retrieve node performs semantic search using sentence-transformer embeddings. Unlike keyword matching, this captures meaning: “my plants look weak and pale” matches nitrogen deficiency even though the words “nitrogen” and “deficiency” don’t appear in the query. This approach aligns with recent advances in retrieval-augmented generation (Lewis et al., 2020).

The check confidence node implements conditional routing based on retrieval scores and input clarity. Scores above 0.35 route to synthesis. Explicit crop-name-only inputs route to

clarification regardless of score (because matching “maize” to random maize entries is noise, not signal). Scores below 0.35 route to refusal.

The synthesize node generates structured responses constrained to retrieved KB entries. It produces five sections (Likely Issue, Why This May Be Happening, What to Check Next, Suggested Action, When to Seek Local Support) plus an optional “AI Additional Context” section that is explicitly labeled as not from the verified knowledge base.

This architecture reflects a genuine multi-step reasoning process where each node makes a real decision. This is fundamentally different from a single prompt chatbot that generates an unchecked response.

5. Responsible AI Considerations

AgiriteChat is designed with safety guardrails appropriate for agricultural advisory, where incorrect recommendations can cause economic harm to vulnerable farmers:

Confidence gating ensures the system never guesses on crop decisions. The refusal node explicitly states the system’s inability to provide a reliable answer and directs farmers to local extension services. This conservative approach prioritizes honesty over helpfulness, consistent with responsible AI frameworks that emphasize transparency and user safety (Jobin et al., 2019).

Source separation provides transparency by visually distinguishing verified knowledge base content (green “From knowledge base” badge with source name) from AI-generated guidance (amber “Additional AI guidance” badge with explicit disclaimer). Farmers can judge for themselves what is verified vs. interpreted.

The vision pipeline separates observation from diagnosis. The vision model is instructed to describe symptoms (leaf color, lesion patterns, pest damage) without attempting to name

diseases. This prevents the common failure mode of vision models confidently identifying conditions they were not trained on.

Hard rules constrain the generative model: never provide specific pesticide or fertilizer doses, never invent information not in the knowledge base, always recommend extension for severe issues. Every advisory includes a “When to seek local support” section.

The zero-cost deployment (Groq free tier, Streamlit Cloud free tier, Open-Meteo free API) demonstrates that responsible AI tools do not require significant financial investment, which is relevant for developing-country agricultural contexts.

6. Innovation and Creativity

Several aspects of AgiriteChat are genuinely uncommon in agricultural AI tools, particularly at the student prototype level:

The confidence gating with explicit refusal path is the single most unusual feature. Almost every AI tool is designed to always respond. AgiriteChat has a hardcoded path where the agent refuses and escalates to human expertise. Building a system that deliberately says “I don’t know” is rare because it feels like a product failure, but in agriculture it is a safety feature.

The separation of vision observation from diagnosis prevents hallucination. Most photo-based agricultural AI systems attempt to directly identify diseases from images. AgiriteChat’s vision model only describes what it sees, and the diagnosis comes from the knowledge base through the standard retrieval pipeline.

Native multilingual generation (not post-translation) produces more natural language. The system includes Kinyarwanda (a low-resource language) and honestly labels it as beta.

Voice input via Whisper addresses accessibility for low-literacy farmers who may find text input challenging.

The entire system was built through a web browser (GitHub web interface) without local development tools, demonstrating the democratization of AI development.

7. User Experience and Usability

AgiriteChat was designed with the end user where smallholder farmer are at the center of every design decision. The user interface uses an editorial aesthetic with a warm color palette (cream, forest green, terracotta) that feels approachable rather than clinical. Typography uses Fraunces serif for headings and DM Sans for body text, creating visual hierarchy without overwhelming the user.

The interaction is organized into three tabs (Ask a question, upload a photo, Knowledge library) placed at the top of the page so farmers see them immediately without scrolling. Quick start preset buttons allow one-tap access to common problems like “Maize leaf blight” or “Soybean nitrogen.”

Farmer profiles allow session-based personalization: region, crops grown, farm size, and planting date. This context is passed to the agent for more relevant recommendations.

The structured 5-section answer format (Likely Issue, Why, What to Check, Action, When to Seek Support) provides clear, actionable guidance rather than unstructured paragraphs. Confidence badges and source attribution give farmers the information they need to judge the reliability of each response.

Voice input and four-language support address accessibility barriers. Weather display in the sidebar provides contextual awareness without requiring the farmer to switch applications.

8. Results and Demonstration

AgiriteChat was successfully deployed at agiritechat.streamlit.app. Key demonstration results:

Text advisory: “My maize leaves are turning yellow from the bottom” correctly identified nitrogen deficiency at high confidence (0.87), with source attribution to Rwanda Agriculture Board Fertilizer Guide.

Photo diagnosis: A real maize leaf photo with brown lesions was correctly processed through the vision pipeline, extracting symptoms (brown lesions, irregular shape) and matching to fungal disease entries at 0.89 confidence.

Greeting detection: “Hi” correctly returns a conversational welcome instead of a fake advisory. “maize” alone correctly asks for symptoms instead of generating generic content.

Multilingual: The full UI and agent responses translate across English, Kiswahili, French, and Kinyarwanda.

Source separation: Advisory responses display green “From knowledge base” badges with source names, and amber “Additional AI guidance” badges when the LLM adds context beyond the KB.

Voice input: Groq Whisper transcribes spoken questions and routes them through the standard agent pipeline.

Weather: Current conditions and 3-day forecast display in the sidebar for the farmer’s region.

9. Limitations and Future Work

- The knowledge base contains 101 entries, which limits coverage. Expansion to more than 1000 entries would improve confidence scores and reduce refusals.

- The Streamlit interface cannot provide persistent accounts, push notifications, or offline access. A React Native mobile app would address these limitations.
- SQLite feedback logging is ephemeral on Streamlit Cloud. A persistent database (Supabase PostgreSQL) would enable long-term performance tracking.
- Kinyarwanda language quality is limited by Llama 3.3 70B’s training data for this low-resource language.
- Weather data is display-only and does not yet influence recommendations. Weather-aware advisory (e.g., fungicide timing based on rainfall) is a natural next step.
- The system has not been tested with actual smallholder farmers. Field validation with extension officers is essential before any real-world deployment.

10. Conclusion

This project demonstrates how agentic AI systems can improve access to agricultural decision support when designed with appropriate safety guardrails. The LangGraph state machine with confidence gating, source separation, and grounded synthesis moves beyond simple chatbot interaction to genuine agentic behavior where the system classifies, retrieves, evaluates, and decides before generating a response. The zero-cost deployment on Groq and Streamlit Cloud demonstrates that meaningful AI-powered agricultural tools can be built without significant financial investment. The system’s refusal to guess on crop decisions, its transparent source attribution, and its separation of vision observation from diagnosis represent responsible AI design choices that prioritize farmer safety over system convenience. AgiriteChat is not a replacement for human agronomic expertise. It is a tool that meets farmers where they are, in the language they speak, with honesty about what it knows and what it does not.

References

- Swanson, B. E., & Davis, K. (2014). *Status of agricultural extension and rural advisory services worldwide: Summary report*. Global Forum for Rural Advisory Services. https://www.g-fras.org/images/wwes/GFRAS-Status_of_Rural_Advisory_Services_Worldwide.pdf
- Jobin, A., Ienca, M., & Vayena, E. (2019). The global landscape of AI ethics guidelines. *Nature Machine Intelligence*, 1(9), 389–399. <https://doi.org/10.1038/s42256-019-0088-2>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>